

Library Management System - Homework Project

Overview

In this homework, you will build a **Library Management System** using FastAPI. You will create two main entities - **Books** and **Members** - with full CRUD, sorting, search, and pagination, following the same patterns as the Items CRUD project.

By the end, your library app will have:

- A **dashboard** showing overall library statistics
 - **Book management** (CRUD + sort/search/pagination by ISBN, title, author, year)
 - **Member management** (CRUD + sort/search/pagination by Member ID, name, email)
 - Separate routers, templates, and JSON data files for each entity
-

Step 0 - Project Setup & Git Init

0.1 Create the project folder and initialise with uv

```
cd /path/to/your/workspace  
mkdir library-management  
cd library-management
```

Initialise the project with uv:

```
uv init
```

This creates a `pyproject.toml` and a basic project skeleton.

0.2 Create a virtual environment and activate it

```
uv venv
```

Activate it:

- **Windows (PowerShell):**

```
.venv\Scripts\Activate.ps1
```

- **Windows (CMD):**

```
.venv\Scripts\activate.bat
```

- **macOS / Linux:**

```
source .venv/bin/activate
```

0.3 Install dependencies

```
uv pip install fastapi uvicorn[standard]
```

0.4 Create the project directory structure

```
library-management/
├── app/
│   ├── __init__.py
│   ├── main.py                # FastAPI app entry point
│   ├── routes/
│   │   ├── __init__.py
│   │   ├── books.py          # Book routes
│   │   └── members.py        # Member routes
│   ├── static/
│   │   └── css/
│   │       ├── custom.css    # Copy from Items CRUD project
│   │       └── fa.min.css     # Copy from Items CRUD project
│   └── templates/
│       ├── base.html         # Base layout (copy and adapt)
│       ├── landing.html      # Dashboard / main landing page
│       ├── books/
│       │   ├── books_landing.html # Book list + sort/search/pagination
│       │   ├── book_form.html    # Add / Edit book form
│       │   └── book_delete.html  # Delete confirmation
│       └── members/
│           ├── members_landing.html # Member list + sort/search/pagination
│           ├── member_form.html     # Add / Edit member form
│           └── member_delete.html   # Delete confirmation
├── books.json                # Book data file
└── members.json              # Member data file
```

Create all directories:

```
mkdir -p app/routes app/static/css app/templates/books app/templates/members
```

Create empty `__init__.py` files using VS Code (right-click in the Explorer panel → New File → name it `__init__.py`) in both `app/` and `app/routes/`. These files can be completely empty - they tell Python that these folders are packages.

0.5 Copy static files from the Items CRUD project

Copy these two files from your completed Items CRUD project:

- `my_crud_app/app/static/css/fa.min.css` → `library-management/app/static/css/fa.min.css`
- `my_crud_app/app/static/css/custom.css` → `library-management/app/static/css/custom.css`

0.6 Create the JSON data files

books.json (sample data):

```
[
  {
    "isbn": "978-0-7475-3269-9",
    "title": "Harry Potter and the Philosopher's Stone",
    "author": "J.K. Rowling",
    "year": 1997,
    "is_loaned": false,
    "loaned_to": null
  },
  {
    "isbn": "978-0-061-12408-4",
    "title": "The Hobbit",
    "author": "J.R.R. Tolkien",
    "year": 1937,
    "is_loaned": true,
    "loaned_to": "M001"
  },
  {
    "isbn": "978-0-452-28423-4",
    "title": "1984",
    "author": "George Orwell",
    "year": 1949,
    "is_loaned": false,
    "loaned_to": null
  },
  {
    "isbn": "978-0-14-103614-4",
    "title": "To Kill a Mockingbird",
    "author": "Harper Lee",
    "year": 1960,
    "is_loaned": true,
    "loaned_to": "M003"
  },
  {
    "isbn": "978-0-7432-7356-5",
```

```
"title": "The Great Gatsby",
"author": "F. Scott Fitzgerald",
"year": 1925,
"is_loaned": false,
"loaned_to": null
},
{
  "isbn": "978-0-307-27767-1",
  "title": "The Catcher in the Rye",
  "author": "J.D. Salinger",
  "year": 1951,
  "is_loaned": false,
  "loaned_to": null
},
{
  "isbn": "978-0-316-76948-0",
  "title": "The Lord of the Rings",
  "author": "J.R.R. Tolkien",
  "year": 1954,
  "is_loaned": true,
  "loaned_to": "M002"
},
{
  "isbn": "978-0-14-118776-1",
  "title": "Animal Farm",
  "author": "George Orwell",
  "year": 1945,
  "is_loaned": false,
  "loaned_to": null
},
{
  "isbn": "978-0-06-112008-4",
  "title": "Brave New World",
  "author": "Aldous Huxley",
  "year": 1932,
  "is_loaned": false,
  "loaned_to": null
},
{
  "isbn": "978-0-14-044926-6",
  "title": "The Odyssey",
  "author": "Homer",
  "year": -800,
  "is_loaned": true,
  "loaned_to": "M001"
},
{
  "isbn": "978-0-679-72316-5",
  "title": "One Hundred Years of Solitude",
  "author": "Gabriel Garcia Marquez",
  "year": 1967,
  "is_loaned": false,
  "loaned_to": null
},
},
```

```
{
  "isbn": "978-0-14-143951-8",
  "title": "Pride and Prejudice",
  "author": "Jane Austen",
  "year": 1813,
  "is_loaned": false,
  "loaned_to": null
},
{
  "isbn": "978-0-14-103614-4",
  "title": "The Hitchhiker's Guide to the Galaxy",
  "author": "Douglas Adams",
  "year": 1979,
  "is_loaned": false,
  "loaned_to": null
},
{
  "isbn": "978-0-385-47454-6",
  "title": "The Da Vinci Code",
  "author": "Dan Brown",
  "year": 2003,
  "is_loaned": false,
  "loaned_to": null
},
{
  "isbn": "978-0-432-14783-6",
  "title": "Fahrenheit 451",
  "author": "Ray Bradbury",
  "year": 1953,
  "is_loaned": false,
  "loaned_to": null
}
]
```

members.json (sample data):

```
[
  {
    "member_id": "M001",
    "name": "Alice Johnson",
    "email": "alice@example.com",
    "phone": "555-0101"
  },
  {
    "member_id": "M002",
    "name": "Bob Smith",
    "email": "bob@example.com",
    "phone": "555-0102"
  },
  {
    "member_id": "M003",
    "name": "Carol Williams",

```

```
    "email": "carol@example.com",
    "phone": "555-0103"
  },
  {
    "member_id": "M004",
    "name": "David Brown",
    "email": "david@example.com",
    "phone": "555-0104"
  },
  {
    "member_id": "M005",
    "name": "Eve Davis",
    "email": "eve@example.com",
    "phone": "555-0105"
  },
  {
    "member_id": "M006",
    "name": "Frank Miller",
    "email": "frank@example.com",
    "phone": "555-0106"
  }
]
```

Note on identifiers: Books use their **ISBN** (a unique string like 978-0-7475-3269-9) instead of a numeric index. Members use a **Member ID** (a string like M001) instead of `loop.index0`. This is the key difference from the Items CRUD project - you will use these string identifiers in URLs (`/books/{isbn}`, `/members/{member_id}`) and in edit/delete links.

0.7 Initialise Git and make the first commit

```
git init
git add -A
git commit -m "Step 0: Project setup with uv, folder structure, static files, and sample data"
```

Step 1 - Base Template, App Entry Point & Dashboard

1.1 Create `app/templates/base.html`

Create a base layout similar to the Items CRUD project. It should include:

- FontAwesome (`/static/css/fa.min.css`) and custom CSS (`/static/css/custom.css`)
- A navigation bar with links to: **Dashboard** (`/`), **Books** (`/books/landing`), **Members** (`/members/landing`)
- A content block `{% block content %}` for child templates

Reference: Look at `my_crud_app/app/templates/base.html` for the exact structure.

Skeleton:

```

<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}Library Manager{% endblock %}</title>
  <link rel="stylesheet" href="/static/css/fa.min.css">
  <link rel="stylesheet" href="/static/css/custom.css">
</head>
<body>

<nav class="navbar">
  <div class="container">
    <a href="/" class="navbar-brand">
      <i class="fas fa-book"></i> Library Manager
    </a>
    <ul class="navbar-nav">
      <li><a href="/"><i class="fas fa-chart-pie"></i> Dashboard</a></li>
      <li><a href="/books/landing"><i class="fas fa-book-open"></i>
Books</a></li>
      <li><a href="/members/landing"><i class="fas fa-users"></i>
Members</a></li>
    </ul>
  </div>
</nav>

<main class="container" style="margin-top: 2rem;">
  {% block content %}{% endblock %}
</main>

<footer style="text-align: center; margin: 3rem 0 1rem; color: #5c6b7a;">
  <hr>
  <small>&copy; 2026 - Library Management Workshop</small>
</footer>

</body>
</html>

```

1.2 Create `app/main.py`

This is the FastAPI entry point. It must:

1. Mount the `/static` directory
2. Include the books router with prefix `/books`
3. Include the members router with prefix `/members`
4. Have a **root endpoint** `GET /` that renders the dashboard

The root endpoint should compute library statistics from both `books.json` and `members.json`:

- **Total books** - `len(load_books())`
- **Available books** - books where `is_loaned == false`
- **Loaned books** - books where `is_loaned == true`
- **Total members** - `len(load_members())`
- **Authors count** - number of unique authors
- **Oldest book** - the book with the lowest year
- **Newest book** - the book with the highest year

Skeleton for `app/main.py`:

```
from fastapi import FastAPI, Request
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from app.routes.books import router as books_router
from app.routes.members import router as members_router
import json
import os

BOOKS_FILE = "books.json"
MEMBERS_FILE = "members.json"

def load_books():
    if not os.path.exists(BOOKS_FILE):
        return []
    with open(BOOKS_FILE, "r") as f:
        return json.load(f)

def load_members():
    if not os.path.exists(MEMBERS_FILE):
        return []
    with open(MEMBERS_FILE, "r") as f:
        return json.load(f)

app = FastAPI(title="Library Manager")

app.mount("/static", StaticFiles(directory="app/static"), name="static")

app.include_router(books_router, prefix="/books")
app.include_router(members_router, prefix="/members")

templates = Jinja2Templates(directory="app/templates")

@app.get("/")
async def root(request: Request):
    books = load_books()
    members = load_members()
    # --- COMPUTE STATISTICS HERE ---
    # total_books, available_books, loaned_books,
    # total_members, unique_authors, oldest_book, newest_book
    return templates.TemplateResponse(
        request=request,
        name="landing.html",
```

```

        context={...} # pass all computed stats
    )

```

Task: Implement the statistics computation. Hint:

- `available_books = [b for b in books if not b["is_loaned"]]`
- `unique_authors = len(set(b["author"] for b in books))`
- Oldest/newest book: use `min(books, key=lambda b: b["year"]) / max(...)`

1.3 Create `app/templates/landing.html`

This is the dashboard page. It should look like the Items CRUD dashboard but with library statistics.

Skeleton:

```

{% extends "base.html" %}

{% block title %}Dashboard{% endblock %}

{% block content %}
<h2><i class="fas fa-chart-pie"></i> Library Dashboard</h2>

<!-- STATISTICS CARDS -->
<div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 1rem; margin: 1.5rem 0;">
    <div class="card" style="text-align: center;">
        <div class="card-header">Total Books</div>
        <div class="card-body">
            <span style="font-size: 2.5rem; font-weight: 700; color: #2b9348;">{{
total_books }}</span>
        </div>
    </div>
    <div class="card" style="text-align: center;">
        <div class="card-header">Available</div>
        <div class="card-body">
            <span style="font-size: 2.5rem; font-weight: 700; color: #2b9348;">{{
available_books }}</span>
        </div>
    </div>
    <div class="card" style="text-align: center;">
        <div class="card-header">Loaned Out</div>
        <div class="card-body">
            <span style="font-size: 2.5rem; font-weight: 700; color: #e9c46a;">{{
loaned_books }}</span>
        </div>
    </div>
    <div class="card" style="text-align: center;">
        <div class="card-header">Members</div>
        <div class="card-body">
            <span style="font-size: 2.5rem; font-weight: 700; color: #145c32;">{{
total_members }}</span>
        </div>
    </div>
</div>

```

```

        </div>
    </div>
</div>

<!-- quick summary table -->
<div class="card">
    <div class="card-header"><i class="fas fa-list"></i> Summary</div>
    <div class="card-body">
        <table class="table">
            <tbody>
                <tr><td><strong>Total books</strong></td><td>{{ total_books }}
</td></tr>
                <tr><td><strong>Available to loan</strong></td><td>{{
available_books }}</td></tr>
                <tr><td><strong>Currently loaned</strong></td><td>{{ loaned_books
}}</td></tr>
                <tr><td><strong>Total members</strong></td><td>{{ total_members }}
</td></tr>
                <tr><td><strong>Unique authors</strong></td><td>{{ unique_authors
}}</td></tr>
                <tr><td><strong>Oldest book</strong></td><td>"{{ oldest_title }}"
({{ oldest_year }})</td></tr>
                <tr><td><strong>Newest book</strong></td><td>"{{ newest_title }}"
({{ newest_year }})</td></tr>
            </tbody>
        </table>
    </div>
</div>

<!-- action buttons -->
<div style="text-align: center; margin: 2rem 0;">
    <a href="/books/landing" class="btn btn-primary" style="padding: 1rem 2rem;
font-size: 1.2rem;">
        <i class="fas fa-book-open"></i> Manage Books
    </a>
    <a href="/members/landing" class="btn btn-primary" style="padding: 1rem 2rem;
font-size: 1.2rem;">
        <i class="fas fa-users"></i> Manage Members
    </a>
</div>
{% endblock %}

```

1.4 Create empty route files

app/routes/books.py:

```

from fastapi import APIRouter, Request, Query
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from fastapi import Form, status
from fastapi.responses import RedirectResponse

```

```
from fastapi import HTTPException
import json
import os
import math

router = APIRouter()
templates = Jinja2Templates(directory="app/templates")
BOOKS_FILE = "books.json"
ITEMS_PER_PAGE = 5

def load_books():
    if not os.path.exists(BOOKS_FILE):
        return []
    with open(BOOKS_FILE, "r") as f:
        return json.load(f)
```

app/routes/members.py:

```
from fastapi import APIRouter, Request, Query
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from fastapi import Form, status
from fastapi.responses import RedirectResponse
from fastapi import HTTPException
import json
import os
import math

router = APIRouter()
templates = Jinja2Templates(directory="app/templates")
MEMBERS_FILE = "members.json"
ITEMS_PER_PAGE = 5

def load_members():
    if not os.path.exists(MEMBERS_FILE):
        return []
    with open(MEMBERS_FILE, "r") as f:
        return json.load(f)
```

1.5 Test and commit

Run the app:

```
uvicorn app.main:app --reload
```

Visit <http://127.0.0.1:8000/>. You should see the dashboard with statistics.

Commit:

```
git add -A
git commit -m "Step 1: Base template, main.py with dashboard stats, empty route files"
```

Step 2 - Book CRUD (Create, Read, Update, Delete)

Now implement full CRUD for books in `app/routes/books.py`. Follow the Items CRUD pattern exactly, but with these key differences:

Item CRUD	Book CRUD
Numeric <code>id</code> index	ISBN string as identifier
<code>products</code> list in context	<code>books</code> list in context
<code>items.json</code>	<code>books.json</code>
<code>templates/item_form.html</code>	<code>templates/books/book_form.html</code>
<code>templates/delete_confirm.html</code>	<code>templates/books/book_delete.html</code>
<code>templates/items/items_landing.html</code>	<code>templates/books/books_landing.html</code>

2.1 Book listing page - GET `/books/landing`

Skeleton route:

```
@router.get("/landing", response_class=HTMLResponse)
async def books_landing(request: Request):
    books = load_books()
    return templates.TemplateResponse(
        request=request,
        name="books/books_landing.html",
        context={
            "books": books
        }
    )
```

Skeleton template `app/templates/books/books_landing.html`:

```
{% extends "base.html" %}

{% block title %}Books{% endblock %}

{% block content %}
<h2><i class="fas fa-book-open"></i> Books Management</h2>

<!-- ADD BUTTON -->
```

```

<div style="margin: 1.5rem 0;">
  <a href="/books/add" class="btn btn-primary">
    <i class="fas fa-plus"></i> Add New Book
  </a>
</div>

<!-- BOOKS TABLE -->
<div class="card">
  <div class="card-header"><i class="fas fa-table"></i> All Books</div>
  <div class="card-body" style="padding: 0;">
    <table class="table">
      <thead>
        <tr>
          <th>ISBN</th>
          <th>Title</th>
          <th>Author</th>
          <th>Year</th>
          <th>Status</th>
          <th>Actions</th>
        </tr>
      </thead>
      <tbody>
        <!-- TODO: Fill in the for loop here (see Items CRUD project for
reference)
                                Use {{ book.isbn }} for the edit/delete URLs, not loop.index0
                                Show book title, author, year
                                Show "Available" badge if not loaned, "Loaned to X" badge if
loaned
                                Show edit and delete action buttons
                                -->
      </tbody>
    </table>
  </div>
</div>
{% endblock %}

```

Important: Note that we use `{{ book.isbn }}` in edit/delete URLs instead of `loop.index0`. The ISBN is the unique identifier for each book.

2.2 Add book form - GET `/books/add`

Show a form to add a new book. The form must have fields for: `isbn`, `title`, `author`, `year`, and a checkbox for `is_loaned`.

Route:

```

@router.get("/add", response_class=HTMLResponse)
async def add_book_form(request: Request):
    return templates.TemplateResponse(
        request=request,
        name="books/book_form.html",
    )

```

```

        context={"editing": False, "book": None, "book_isbn": None}
    )

```

Template `app/templates/books/book_form.html`: Follow the pattern in `my_crud_app/app/templates/item_form.html`. Use ISBN instead of the price field.

2.3 Create book - POST `/books/`

Process the form submission and add the book to `books.json`.

Important validation: Check if a book with the same ISBN already exists. If it does, return an error message (you can pass an `error` variable to the template).

```

@router.post("/")
async def create_book(
    isbn: str = Form(...),
    title: str = Form(...),
    author: str = Form(...),
    year: int = Form(...),
    is_loaned: bool = Form(False),
    loaned_to: str = Form("")
):
    books = load_books()
    # Check for duplicate ISBN
    for book in books:
        if book["isbn"] == isbn:
            # ISBN already exists - show error
            ...
    new_book = {
        "isbn": isbn,
        "title": title,
        "author": author,
        "year": year,
        "is_loaned": is_loaned,
        "loaned_to": loaned_to if is_loaned else None
    }
    books.append(new_book)
    with open(BOOKS_FILE, "w") as f:
        json.dump(books, f, indent=4)
    return RedirectResponse("/books/landing",
        status_code=status.HTTP_303_SEE_OTHER)

```

2.4 Edit book form - GET `/books/edit/{isbn}`

Find the book by ISBN (not by index!) and pass it to the form template.

```

@router.get("/edit/{book_isbn}", response_class=HTMLResponse)
async def edit_book_form(request: Request, book_isbn: str):
    books = load_books()

```

```

book = None
for b in books:
    if b["isbn"] == book_isbn:
        book = b
        break
if book is None:
    raise HTTPException(status_code=404, detail="Book not found")
return templates.TemplateResponse(
    request=request,
    name="books/book_form.html",
    context={"editing": True, "book": book, "book_isbn": book_isbn}
)

```

2.5 Update book - POST /books/{isbn}

```

@router.post("/{book_isbn}")
async def update_book(
    book_isbn: str,
    title: str = Form(...),
    author: str = Form(...),
    year: int = Form(...),
    is_loaned: bool = Form(False),
    loaned_to: str = Form("")
):
    books = load_books()
    for i, book in enumerate(books):
        if book["isbn"] == book_isbn:
            books[i] = {
                "isbn": book_isbn,
                "title": title,
                "author": author,
                "year": year,
                "is_loaned": is_loaned,
                "loaned_to": loaned_to if is_loaned else None
            }
    with open(BOOKS_FILE, "w") as f:
        json.dump(books, f, indent=4)
    return RedirectResponse("/books/landing",
        status_code=status.HTTP_303_SEE_OTHER)
    raise HTTPException(status_code=404, detail="Book not found")

```

2.6 Delete book - GET /books/delete/{isbn} and POST /books/delete/{isbn}

Follow the same pattern as the Items CRUD project (`confirm_delete` + `delete_item`) but find the book by ISBN instead of by index.

Template `app/templates/books/book_delete.html`: Show the book's ISBN, title, and author. Use `{{ book.isbn }}` in the form action URL.

2.7 Test and commit

Test all CRUD operations:

- Add a new book
- View the list
- Edit the book (change title, author, etc.)
- Delete the book

```
git add -A
git commit -m "Step 2: Book CRUD (create, read, update, delete) with ISBN
identifiers"
```

Step 3 - Book Sorting, Search & Pagination

Now enhance the `GET /books/landing` route to support sorting, search, and pagination - exactly like the Items CRUD project but adapted for books.

3.1 Update `GET /books/landing`

Add query parameters:

```
@router.get("/landing", response_class=HTMLResponse)
async def books_landing(
    request: Request,
    sort: str = Query("default", pattern="^(default|title|author|year)$"),
    page: int = Query(1, ge=1),
    search: str = Query(None)
):
    books = load_books()

    # Step 1: Apply search filter (by title, author, or ISBN)
    if search:
        search_lower = search.lower()
        filtered = []
        for b in books:
            if (search_lower in b["title"].lower() or
                search_lower in b["author"].lower() or
                search_lower in b["isbn"].lower()):
                filtered.append(b)
        books = filtered

    # Step 2: Apply sorting
    if sort == "title":
        books = sorted(books, key=lambda x: x["title"].lower())
    elif sort == "author":
        books = sorted(books, key=lambda x: x["author"].lower())
    elif sort == "year":
        books = sorted(books, key=lambda x: x["year"])

    # Step 3: Pagination
```

```

total_items = len(books)
total_pages = math.ceil(total_items / ITEMS_PER_PAGE)

if page > total_pages and total_pages > 0:
    return HttpResponseRedirect(
        url=f"/books/landing?sort={sort}&page={total_pages}&search={search or ''}",
        status_code=303
    )

start = (page - 1) * ITEMS_PER_PAGE
end = start + ITEMS_PER_PAGE
paginated_books = books[start:end]

return templates.TemplateResponse(
    ...
)

```

3.2 Update the template with sort/search/pagination

Enhance `books/books_landing.html` to include:

1. **Search bar** - a text input that searches by title, author, or ISBN
2. **Sort controls** - buttons for Default, Title, Author, Year
3. **Pagination** - Previous/Next and page numbers (same pattern as Items CRUD)

Search form skeleton (add above the sort controls):

```

<!-- SEARCH BAR -->
<div class="card" style="margin-bottom: 1rem;">
    <div class="card-body">
        <form method="GET" action="/books/landing">
            <div class="d-flex gap-2">
                <input type="text" name="search" placeholder="Search by title,
author, or ISBN..."
                    value="{{ search if search is defined and search is not
none else '' }}"
                    style="flex: 1;">
                <button type="submit" class="btn btn-primary">
                    <i class="fas fa-search"></i> Search
                </button>
                <a href="/books/landing" class="btn btn-secondary">Clear</a>
            </div>
        </form>
    </div>
</div>

```

Sort controls and **pagination** follow the exact same pattern as `templates/items/items_landing.html`. Use the `base_url` trick to preserve search and sort parameters. Remember:

- `base_url` should be `/books/landing?search={{ search }}` when search is active
- `base_url` should be `/books/landing?` when no search

Skeleton for the `base_url` logic:

```
{% set params = "" %}
{% if search is defined and search is not none and search %}
{% set params = "search=" ~ search ~ "&" %}
{% endif %}
{% set base_url = "/books/landing?" ~ params %}
```

3.3 Test and commit

- Test sorting by title, author, year
- Test searching by title, author, and ISBN
- Test pagination with search and sort combined
- Test the Clear button resets everything

```
git add -A
git commit -m "Step 3: Book sorting, search, and pagination"
```

Step 4 - Member Management (CRUD + Sort/Search/Pagination)

Now implement the **Members** section following exactly the same patterns you used for Books.

4.1 Implement `app/routes/members.py`

Add these endpoints:

Method	Path	Purpose
GET	<code>/members/landing</code>	List members with sort/search/pagination
GET	<code>/members/add</code>	Show add member form
POST	<code>/members/</code>	Create new member
GET	<code>/members/edit/{member_id}</code>	Show edit member form
POST	<code>/members/{member_id}</code>	Update member
GET	<code>/members/delete/{member_id}</code>	Show delete confirmation
POST	<code>/members/delete/{member_id}</code>	Delete member

Member fields: `member_id` (unique string like `M007`), `name`, `email`, `phone`

Member ID auto-generation hint: When creating a new member, you can auto-generate the next Member ID:

```
def next_member_id(members):  
    if not members:  
        return "M001"  
    max_num = max(int(m["member_id"][1:]) for m in members)  
    return f"M{max_num + 1:03d}"
```

4.2 Create member templates

Create these files under `app/templates/members/`:

- `members_landing.html` - table with columns: Member ID, Name, Email, Phone, Actions
 - Sort by: default, name, email
 - Search by: name, email, or member_id
 - Pagination: same pattern
- `member_form.html` - form for add/edit
 - When creating, auto-generate the Member ID (show it as read-only)
 - When editing, Member ID is read-only (shown but not changeable)
- `member_delete.html` - confirmation page showing member name and ID

Skeleton for members landing sort/search:

- Sort fields: `name`, `email`
- Search fields: `name`, `email`, `member_id`

4.3 Update the dashboard buttons

Make sure the dashboard landing page has a button to go to `/members/landing` (this should already be there from Step 1).

4.4 Test and commit

Test all CRUD and sort/search/pagination for members:

```
git add -A  
git commit -m "Step 4: Member management with CRUD, sort, search, and pagination"
```

Step 5 - Final Polish & Verification

5.1 Run the full test checklist

- ☐ Dashboard shows correct statistics
- ☐ Dashboard links to books and members work
- ☐ Books: add, edit, delete work with ISBN

- ☐ Books: search by title, author, ISBN
- ☐ Books: sort by title, author, year
- ☐ Books: pagination preserves sort and search
- ☐ Members: add (auto-generates Member ID), edit, delete
- ☐ Members: search by name, email, member ID
- ☐ Members: sort by name, email
- ☐ Members: pagination preserves sort and search
- ☐ Navigation bar has all three links: Dashboard, Books, Members
- ☐ No `loop.index0` used anywhere - all identifiers are ISBN or Member ID

5.2 Verify ISBN uniqueness

The book creation route should reject duplicate ISBNs. Test by trying to add a book with the same ISBN as an existing one.

5.3 Final commit

```
git add -A
git commit -m "Step 5: Final polish - full library management system complete"
```

Reference: Key Code Snippets from Items CRUD Project

Here are the essential patterns you need, all adapted from the Items CRUD project. Refer to the files in `my_crud_app/` for the complete working examples.

Route pattern (from `my_crud_app/app/routes/items.py`)

```
@router.get("/some-page", response_class=HTMLResponse)
async def some_handler(
    request: Request,
    sort: str = Query("default", pattern="^(default|field1|field2)$"),
    page: int = Query(1, ge=1)
):
    data = load_data()
    # filter (optional)
    # sort
    # paginate
    return templates.TemplateResponse(
        request=request,
        name="folder/template.html",
        context={...}
    )
```

Redirect on invalid page (from `items.py`)

```
if page > total_pages and total_pages > 0:
    return HttpResponseRedirect(
        url=f"/some-page?sort={sort}&page={total_pages}",
        status_code=303
    )
```

Pagination slicing (from `items.py`)

```
start = (page - 1) * ITEMS_PER_PAGE
end = start + ITEMS_PER_PAGE
paginated_data = data[start:end]
```

Template base_url pattern (from `items/items_landing.html`)

```
{% set params = "" %}
{% if search is defined and search is not none and search %}
{% set params = "search=" ~ search ~ "&" %}
{% endif %}
{% set base_url = "/your-path?" ~ params %}

<!-- sort controls -->
<a href="{ { base_url }}sort=title&page=1" class="btn btn-sm ..." >Title</a>

<!-- pagination -->
<a href="{ { base_url }}sort={ { current_sort }}&page={ { current_page - 1 }}" ...>
```

Bonus Challenges (Optional)

If you finish early, try these:

1. **Loan tracking:** Add a loan system where borrowing a book updates both the book's `is_loaned/loaned_to` fields and shows the loan on the member's profile page.
2. **Statistics per member:** On the member detail page, show how many books they have currently loaned.
3. **Year validation:** When adding/editing a book, validate that the year is between 1800 and the current year.
4. **Email format validation:** When adding a member, validate the email contains @.
5. **Delete protection:** Prevent deleting a member who currently has loaned books.